

This code is used to connect Google Colab with Google Drive. The command `drive.mount('/content/drive')` establishes a link between the Colab environment and the user's Google Drive storage. Once mounted, all files and folders stored in Google Drive become accessible from Colab. This enables the project to load MRI image datasets, save trained models, store output results, and directly access project files without the need to upload them repeatedly. It provides a convenient and permanent storage solution for managing large datasets and model files used in the brain tumor detection system.

```
from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

This code is used to verify that the MRI dataset is correctly available in the specified directory. The `os` module provides functions for interacting with the operating system and file structure. The variable `path` stores the location of the training dataset in Google Drive. The `os.listdir(path)` function retrieves and displays all files and folders present in that directory. This helps ensure that the dataset has been successfully loaded and allows the developer to view the available tumor categories before proceeding with preprocessing and model training. It is an important validation step that prevents errors caused by incorrect dataset paths or missing files.

```
import os

path = "/content/drive/MyDrive/MRI DATASET/Training"

print(os.listdir(path))

['pituitary', 'notumor', 'meningioma', 'glioma']
```

This code tells the program where the MRI images are stored. The `train_dir` variable contains the location of the training images, which are used to teach the model how to identify different types of brain tumors. The `test_dir` variable contains the location of the testing images, which are used to check how accurately the trained model can make predictions on new MRI images. By setting these paths, the program can easily find and use the required data for training and testing.

```
train_dir = "/content/drive/MyDrive/MRI DATASET/Training"
test_dir = "/content/drive/MyDrive/MRI DATASET/Testing"
```

This code is used to organize the training dataset into a structured format. First, the different brain tumor categories (Glioma, Meningioma, No Tumor, and Pituitary) are defined. The program then goes through each category folder, reads all MRI image filenames, and counts the number of images available. For every image, the corresponding tumor category is assigned and stored in a table. Finally, all category data is combined into a single dataset and the first few records are displayed. This helps in understanding the dataset structure and ensures that the MRI images are correctly grouped according to their respective classes before model training.

```
import pandas as pd
import os

categories = ['glioma', 'meningioma', 'notumor', 'pituitary']

train_data = []

for category in categories:

    folder_path = os.path.join(train_dir, category)

    images = os.listdir(folder_path)

    count = len(images)

    train_data.append(
        pd.DataFrame({
            "Image": images,
            "Category": [category] * count
        })
    )

df = pd.concat(train_data)

print(df.head())
```

	Image	Category
0	Tr-gl_1373.jpg	glioma
1	Tr-gl_1389.jpg	glioma
2	Tr-gl_1371.jpg	glioma
3	Tr-gl_142.jpg	glioma
4	Tr-gl_143.jpg	glioma

This code is used to check whether the specified dataset folder exists in the given location. The `os.path.exists()` function verifies if the folder path is correct and accessible. If the folder is found, the program returns `True`; otherwise, it returns `False`. This step helps

ensure that the MRI image dataset is stored in the correct directory before proceeding with data loading, preprocessing, and model training. It also helps identify path-related errors at an early stage.

```
import os

folder = "/content/drive/MyDrive/MRI DATASET/Training/glioma"

print(os.path.exists(folder))

True
```

This code is used to load the MRI image dataset from the specified folder into TensorFlow. The `image_dataset_from_directory()` function automatically reads the images, organizes them based on their folder names (tumor categories), and creates batches of images for model training. Each image is resized to **224 × 224 pixels** so that all images have the same dimensions. The dataset is divided into batches of **32 images**, which helps the model process the data efficiently during training. This step prepares the MRI images in a format that can be directly used by the Hybrid CNN-EfficientNet model.

```
import tensorflow as tf

dataset = tf.keras.preprocessing.image_dataset_from_directory(
    "/content/drive/MyDrive/MRI DATASET/Training",
    image_size=(224,224),
    batch_size=32
)

Found 5600 files belonging to 4 classes.
```

This code loads the **EfficientNetB0** pre-trained deep learning model, which is used for advanced feature extraction from MRI images. The model uses weights that were previously trained on the ImageNet dataset, allowing it to recognize important image patterns effectively. The parameter `include_top=False` removes the original classification layer because the project uses its own classification layers for brain tumor detection. The input image size is set to **224 × 224 × 3**, which matches the size of the MRI images. By setting `base_model.trainable = False`, the pre-trained layers are frozen so that their learned features are preserved during training. This helps improve classification accuracy while reducing training time and computational requirements.

```
from tensorflow.keras.applications import EfficientNetB0

base_model = EfficientNetB0(
    weights='imagenet',
    include_top=False,
    input_shape=(224,224,3)
)

base_model.trainable = False

Downloading data from https://storage.googleapis.com/keras-applications/efficientnetb0\_notop.h5
16705208/16705208 ————— 0s 0us/step
```

Double-click (or enter) to edit

This code creates the **Hybrid CNN-EfficientNet model** used for brain tumor classification. First, an input layer is defined to accept MRI images of size **224 × 224 × 3**. The CNN part of the model consists of convolution and max-pooling layers that extract basic image features such as edges, shapes, and textures. The extracted features are then converted into a one-dimensional vector using a flatten layer.

At the same time, the EfficientNetB0 model is used to extract advanced and high-level features from the same MRI image. Since EfficientNet is pre-trained on a large image dataset, it can identify complex patterns effectively. The EfficientNet layers are frozen to preserve their learned knowledge and reduce training time.

The features obtained from both CNN and EfficientNet are combined using a concatenation layer. This combined feature set is passed through a dense layer that learns important information for classification. Finally, a Softmax output layer with four neurons predicts one of the four classes: **Glioma, Meningioma, Pituitary Tumor, or No Tumor**.

By combining CNN and EfficientNet, the model benefits from both local feature extraction and advanced feature learning, resulting in improved classification accuracy and better overall performance.

```
from tensorflow.keras.layers import *
from tensorflow.keras.models import Model
from tensorflow.keras.applications import EfficientNetB0

input_layer = Input(shape=(224,224,3))

# CNN
x = Conv2D(32, (3,3), activation='relu')(input_layer)
x = MaxPooling2D()(x)

x = Conv2D(64, (3,3), activation='relu')(x)
x = MaxPooling2D()(x)
```

```

x = Flatten()(x)

# EfficientNet
efficientnet = EfficientNetB0(
    weights='imagenet',
    include_top=False,
    input_shape=(224,224,3)
)

efficientnet.trainable = False

y = efficientnet(input_layer)

y = GlobalAveragePooling2D()(y)

# Combine
combined = concatenate([x,y])

z = Dense(128, activation='relu')(combined)

output = Dense(4, activation='softmax')(z)

model = Model(inputs=input_layer, outputs=output)

```

This code is used to configure the Hybrid CNN-EfficientNet model before training. The **Adam optimizer** is selected because it adjusts the learning process automatically and helps the model achieve better performance. The **sparse categorical cross-entropy loss function** is used since the dataset contains multiple classes represented by numerical labels. The **accuracy metric** is included to measure how correctly the model classifies MRI images during training and testing. This compilation step prepares the model for learning and ensures that its performance can be evaluated effectively throughout the training process.

```

model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

```

This code is used to train the Hybrid CNN-EfficientNet model using the MRI image dataset. The `model.fit()` function starts the learning process by feeding the dataset to the model. During training, the model analyzes the MRI images, learns important features, and adjusts its internal parameters to improve classification accuracy. The parameter `epochs=10` means that the entire dataset is processed ten times, allowing the model to gradually improve its predictions. The training results, such as accuracy and loss values for each epoch, are stored in the variable `history`, which can later be used to generate performance graphs and evaluate the model's learning progress.

```

history = model.fit(dataset, epochs=10)

```

```

Epoch 1/10
175/175 ————— 1066s 6s/step - accuracy: 0.6293 - loss: 79.0586
Epoch 2/10
175/175 ————— 1032s 6s/step - accuracy: 0.9166 - loss: 0.2425
Epoch 3/10
175/175 ————— 1012s 6s/step - accuracy: 0.9664 - loss: 0.1025
Epoch 4/10
175/175 ————— 1049s 6s/step - accuracy: 0.9895 - loss: 0.0402
Epoch 5/10
175/175 ————— 1024s 6s/step - accuracy: 0.9841 - loss: 0.0505
Epoch 6/10
175/175 ————— 1025s 6s/step - accuracy: 0.9909 - loss: 0.0343
Epoch 7/10
175/175 ————— 1003s 6s/step - accuracy: 0.9907 - loss: 0.0308
Epoch 8/10
175/175 ————— 1048s 6s/step - accuracy: 0.9902 - loss: 0.0325
Epoch 9/10
175/175 ————— 990s 6s/step - accuracy: 0.9979 - loss: 0.0098
Epoch 10/10
175/175 ————— 1010s 6s/step - accuracy: 0.9971 - loss: 0.0168

```

This code is used to evaluate the performance of the trained Hybrid CNN-EfficientNet model. The `model.evaluate()` function tests the model on the dataset and calculates important performance metrics such as loss and accuracy. The loss value indicates how far the model's predictions are from the actual results, while the accuracy value shows the percentage of MRI images that were classified correctly. The `print()` statement displays the accuracy in percentage format, making it easier to understand the overall performance of the model. This step helps determine how effectively the model can detect and classify different types of brain tumors from MRI images.

```

loss, accuracy = model.evaluate(dataset)

print(f"Accuracy: {accuracy*100:.2f}%")

175/175 ————— 602s 3s/step - accuracy: 0.9993 - loss: 0.0030
Accuracy: 99.93%

```

This code is used to analyze and visualize the distribution of MRI images across different tumor categories in the training dataset. First, the program accesses the training dataset folder and reads all category folders. It then counts the number of images present in each category, such as Glioma, Meningioma, Pituitary, and No Tumor. The image counts are stored and displayed to provide an overview of the dataset composition.

After counting the images, a bar chart is generated using Matplotlib. The graph displays the tumor categories on the horizontal axis and the corresponding number of images on the vertical axis. This visualization helps identify whether the dataset is balanced or imbalanced and provides useful insights before training the model. Understanding the dataset distribution is important because it can affect the model's learning process and classification performance.

```
import os
import matplotlib.pyplot as plt

# Training dataset path
train_dir = "/content/drive/MyDrive/MRI DATASET/Training"

# Store class names and counts
classes = []
counts = []

# Count images in each class
for folder in os.listdir(train_dir):

    folder_path = os.path.join(train_dir, folder)

    image_count = len(os.listdir(folder_path))

    classes.append(folder)

    counts.append(image_count)

    print(f"{folder}: {image_count}")

# Plot graph
plt.figure(figsize=(8,5))

plt.bar(classes, counts)

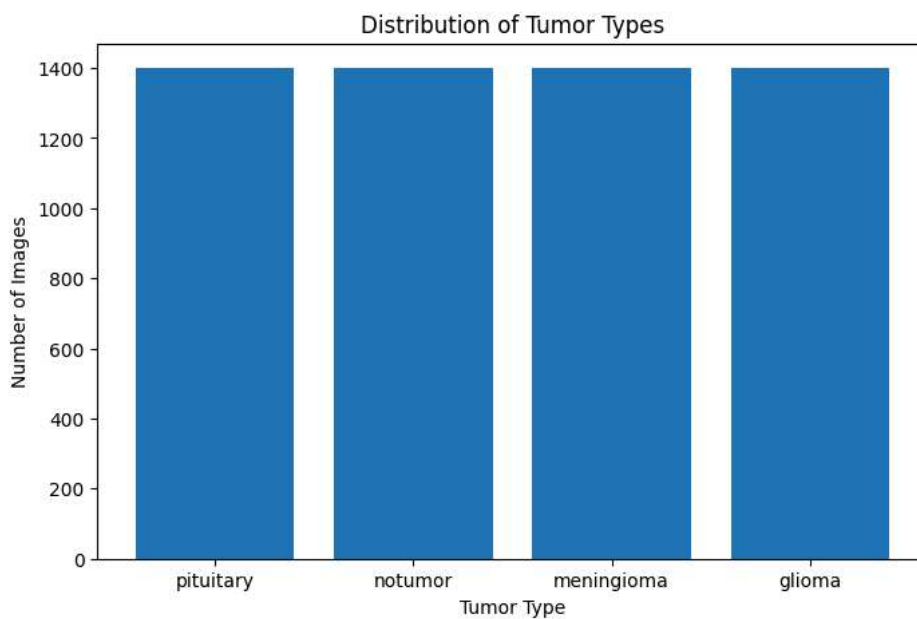
plt.title("Distribution of Tumor Types")

plt.xlabel("Tumor Type")

plt.ylabel("Number of Images")

plt.show()
```

```
pituitary: 1400
notumor: 1400
meningioma: 1400
glioma: 1400
```



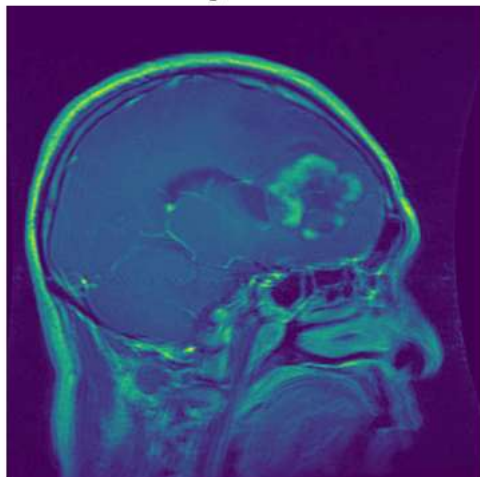
This code is used to display sample MRI images from each tumor category in the dataset. The program creates a figure and selects one image from each category folder, including Glioma, Meningioma, No Tumor, and Pituitary. Each selected image is read and displayed in a separate subplot along with its corresponding category name.

The purpose of this visualization is to provide a quick overview of the dataset and verify that the images are correctly organized into their respective classes. By examining these sample images, the researcher can understand the visual differences between tumor types and

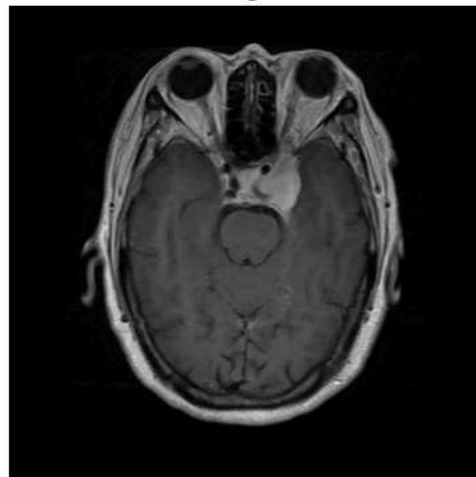
ensure that the dataset has been loaded properly before proceeding with preprocessing and model training. This step also helps in identifying any incorrectly labeled or corrupted images within the dataset.

```
# Visualize sample images for each tumor type
plt.figure(figsize=(12, 8))
for i, category in enumerate(categories):
    folder_path = os.path.join(train_dir, category)
    image_path = os.path.join(folder_path, os.listdir(folder_path)[0])
    img = plt.imread(image_path)
    plt.subplot(2, 2, i+1)
    plt.imshow(img)
    plt.title(category)
    plt.axis("off")
plt.tight_layout()
plt.show()
```

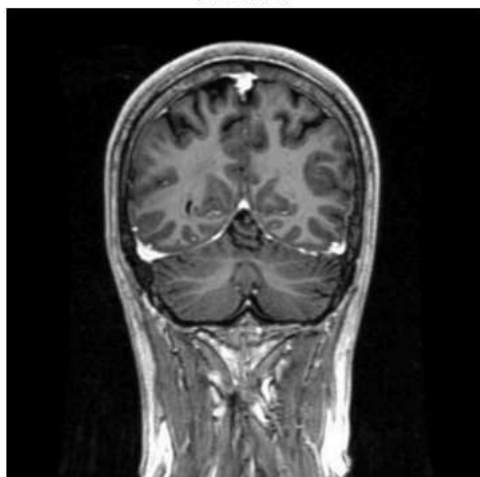
glioma



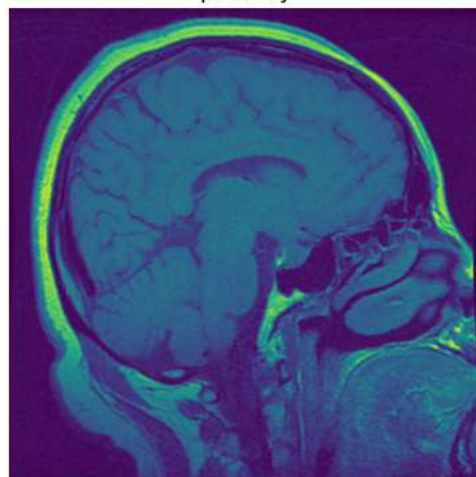
meningioma



notumor



pituitary

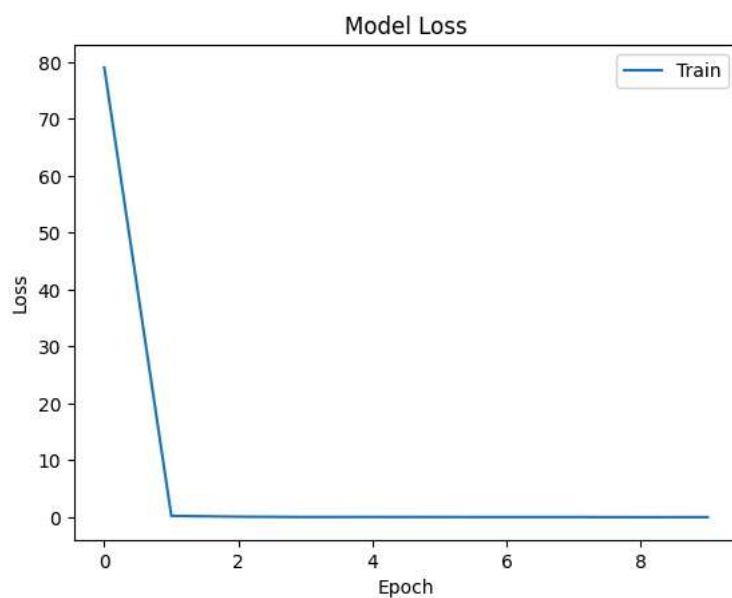


This code is used to visualize the loss of the Hybrid CNN-EfficientNet model during training. The loss values stored in the `history` object are plotted against the number of training epochs. The graph shows how the model's error changes as training progresses.

The horizontal axis represents the epochs, while the vertical axis represents the loss value. A decreasing loss curve indicates that the model is learning effectively and improving its predictions over time. By analyzing this graph, it is possible to determine whether the model is converging properly and to identify potential issues such as underfitting or overfitting. This visualization helps evaluate the overall learning performance of the brain tumor classification model.

```
# Plot the training and validation loss over epochs
plt.plot(history.history['loss'])

plt.title('Model Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend(['Train', 'Validation'])
plt.show()
```



This code is used to evaluate the performance of the trained Hybrid CNN-EfficientNet model on the testing dataset. First, the test MRI images are loaded and normalized by scaling pixel values between 0 and 1. The trained model then makes predictions for all test images and determines the most likely tumor category for each image.

A confusion matrix is generated to compare the actual tumor classes with the predicted classes. The confusion matrix is displayed as a heatmap, providing a clear visual representation of correct and incorrect classifications for each category. This helps in assessing the classification performance of the model and identifying any misclassified cases.

In addition, the code randomly selects a set of MRI images from the testing dataset and displays them along with their predicted and actual labels. This visual comparison allows for manual verification of the model's predictions and provides a better understanding of its performance on unseen data.

Overall, this evaluation process helps measure the accuracy and reliability of the brain tumor detection system and validates its effectiveness in classifying MRI images into Glioma, Meningioma, Pituitary Tumor, and No Tumor categories.

```
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt
import seaborn as sns
import os
from tensorflow.keras.preprocessing.image import ImageDataGenerator

# Test data preprocessing
test_datagen = ImageDataGenerator(rescale=1./255)

# Load test dataset
test_generator = test_datagen.flow_from_directory(
    test_dir,                # path to test folder
    target_size=(224, 224),
    batch_size=32,
    class_mode='categorical',
    shuffle=False
)# Make predictions on the test dataset
predictions = model.predict(test_generator)
print(predictions)
predicted_categories = np.argmax(predictions, axis=1)
print(predicted_categories)
true_categories = test_generator.classes

# Create a confusion matrix
confusion_matrix = tf.math.confusion_matrix(true_categories, predicted_categories)

# Plot the confusion matrix
plt.figure(figsize=(8, 6))
sns.heatmap(confusion_matrix, annot=True, fmt="d", cmap="Blues")
plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("True")
plt.xticks(ticks=np.arange(len(categories)), labels=categories)
plt.yticks(ticks=np.arange(len(categories)), labels=categories)
plt.show()

# Plot sample images with their predicted and true labels
test_images = test_generator.files
sample_indices = np.random.choice(range(len(test_images)), size=9, replace=False)
sample_images = [test_images[i] for i in sample_indices]
sample_predictions = [categories[predicted_categories[i]] for i in sample_indices]
sample_true_labels = [categories[true_categories[i]] for i in sample_indices]
```

```
plt.figure(figsize=(12, 8))
for i in range(9):
    plt.subplot(3, 3, i+1)
    img = plt.imread(os.path.join(test_dir, sample_images[i]))
    plt.imshow(img)
    plt.title(f"Predicted: {sample_predictions[i]}\nTrue: {sample_true_labels[i]}")
    plt.axis("off")
plt.tight_layout()
plt.show()
```

[Show hidden output](#)

This code is used to calculate and analyze the classification performance of the trained Hybrid CNN-EfficientNet model. Using the confusion matrix, the program computes three important evaluation metrics: Precision, Recall, and F1-Score for each tumor category.

Precision measures how many of the images predicted as a particular tumor type were actually correct. Recall measures how effectively the model identifies all images belonging to a specific tumor category. The F1-Score provides a balanced measure by combining both Precision and Recall into a single value. These metrics are calculated separately for Glioma, Meningioma, Pituitary Tumor, and No Tumor classes and are displayed for performance assessment.

The code also visualizes a set of sample MRI images from the testing dataset along with their predicted and actual labels. If the prediction matches the true label, the title is displayed in green, indicating a correct classification. If the prediction is incorrect, the title appears in red, highlighting a misclassification. This visual analysis helps evaluate the model's prediction accuracy and provides a clear understanding of how well the system performs on unseen MRI images.

Overall, this evaluation step validates the effectiveness, reliability, and classification capability of the proposed brain tumor detection system.

```
# Calculate precision, recall, and F1-score from the confusion matrix
precision = np.diag(confusion_matrix) / np.sum(confusion_matrix, axis=0)
recall = np.diag(confusion_matrix) / np.sum(confusion_matrix, axis=1)
f1_score = 2 * (precision * recall) / (precision + recall)

# Print precision, recall, and F1-score for each class
for i, category in enumerate(categories):
    print(f"Class: {category}")
    print(f"Precision: {precision[i]}")
    print(f"Recall: {recall[i]}")
    print(f"F1-Score: {f1_score[i]}")
    print()

# Analyze the sample images and their predictions
plt.figure(figsize=(12, 8))
for i in range(9):
    plt.subplot(3, 3, i+1)
    img = plt.imread(os.path.join(test_dir, sample_images[i]))
    plt.imshow(img)
    if sample_predictions[i] == sample_true_labels[i]:
        plt.title(f"Predicted: {sample_predictions[i]}\nTrue: {sample_true_labels[i]}", color='green')
    else:
        plt.title(f"Predicted: {sample_predictions[i]}\nTrue: {sample_true_labels[i]}", color='red')
    plt.axis("off")
plt.tight_layout()
plt.show()
```

```

/tmp/ipykernel_8609/3415192788.py:2: RuntimeWarning: invalid value encountered in divide
  precision = np.diag(confusion_matrix) / np.sum(confusion_matrix, axis=0)
Class: glioma
Precision: 0.10204081632653061
Recall: 0.0125
F1-Score: 0.022271714922049

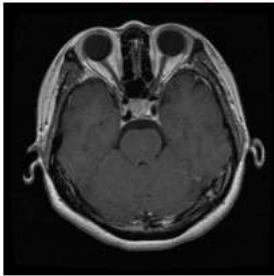
Class: meningioma
Precision: 0.2905982905982906
Recall: 0.34
F1-Score: 0.3133640552995392

Class: notumor
Precision: 0.29916897506925205
Recall: 0.81
F1-Score: 0.43695212407282535

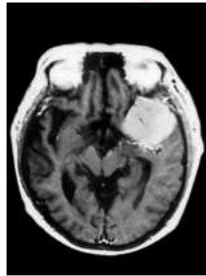
Class: pituitary
Precision: nan
Recall: 0.0
F1-Score: nan

```

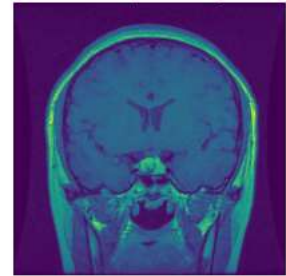
Predicted: notumor
True: pituitary



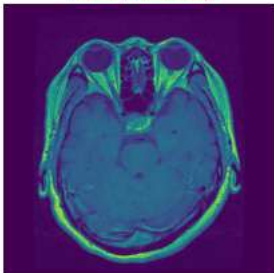
Predicted: notumor
True: meningioma



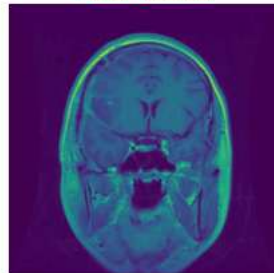
Predicted: notumor
True: pituitary



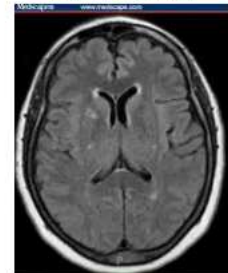
Predicted: notumor
True: pituitary



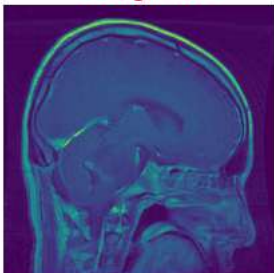
Predicted: notumor
True: glioma



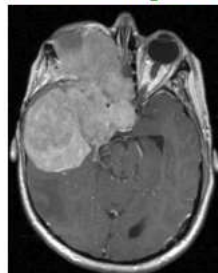
Predicted: notumor
True: notumor



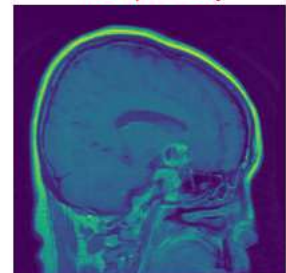
Predicted: meningioma
True: glioma



Predicted: meningioma
True: meningioma



Predicted: meningioma
True: pituitary



This code is used to save the trained Hybrid CNN-EfficientNet model after the training process is completed. The `model.save()` function stores the entire model, including its architecture, learned weights, and training configuration, into a file named **model.h5**. Saving the model allows it to be reused later without retraining from scratch.

The saved model can be loaded whenever required for predicting brain tumor categories from new MRI images. This reduces computation time and makes deployment of the brain tumor detection system easier. The model file serves as the final output of the training phase and is an essential component for developing web applications and real-time prediction systems.

```

# Save the trained model
model.save("model.h5")

```

WARNING:abs1:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format is consid

IT REPRESENTS ACCURACY GRAPH

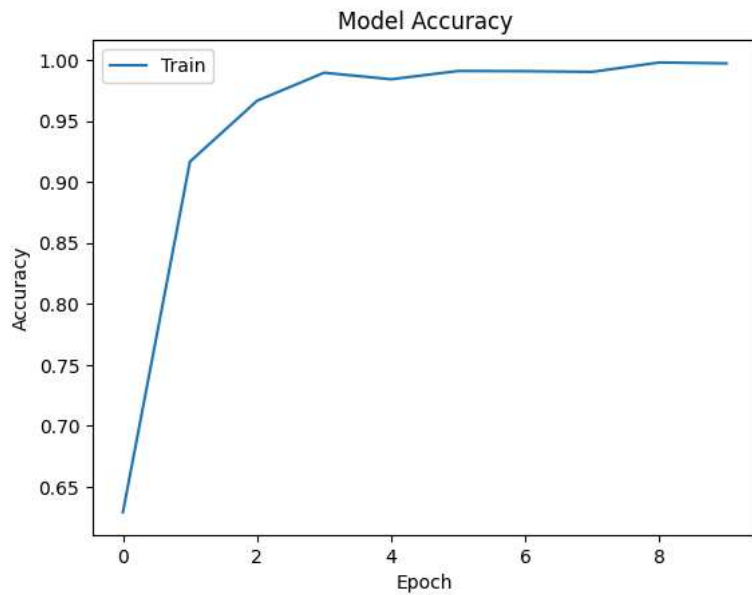
```

# Plot the training and validation accuracy over epochs
plt.plot(history.history['accuracy'])

plt.title('Model Accuracy')
plt.xlabel('Epoch')

```

```
plt.ylabel('Accuracy')
plt.legend(['Train', 'Validation'])
plt.show()
```



PREDICTING THE MODEL

```
import numpy as np
from tensorflow.keras.preprocessing.image import load_img, img_to_array

# Define class names
CLASS_NAMES = ['glioma', 'meningioma', 'notumor', 'pituitary']

# Load image
img = load_img(
    '/content/drive/MyDrive/Hirnmetastase_MRT-T1_KM.jpg',
    target_size=(224,224)
)

# Convert image to array
img = img_to_array(img)

# Normalize
img = img / 255.0

# Add batch dimension
img = np.expand_dims(img, axis=0)

# Predict
prediction = model.predict(img)

# Get predicted class
predicted_class = CLASS_NAMES[np.argmax(prediction)]

print("Predicted Tumor:", predicted_class)
```

1/1 ————— 0s 146ms/step
Predicted Tumor: meningioma